Python Programming
Long Weekend Practice Assignment

Covers everything from Day 2 through Day 6

Name: _____	Batch: _____	Total Score: _____ / 100

Section	Topics Covered	Days	Points
A — Web Fundamentals	DNS, IP, Domain, HTTP, API, Endpoints	Day 2	15
B — Python Basics	Data types, variables, input, type conv.	Days 3-4	15
C — Operators & Logic	Arithmetic, comparison, logical, if/elif	Days 4-5	15
D — Loops	for, while, break, continue, range()	Day 5	15
E — Data Structures	List, Tuple, Set, Dictionary	Days 5-6	20
F — Functions	def, parameters, return, reusability	Day 6	10
G — Mini Project	Combine everything into one program	Days 2-6	10

Instructions: Answer all questions in VS Code. Save each section as a separate .py file (e.g. section_a.py, section_b.py). Push all files to your GitHub repo. Write clean, commented code. Test every edge case before submitting.

Section A — Web Fundamentals

Day 2 | 15 pts

Q1. DNS & Domain Name Resolution

THEORY

5 pts

Answer the following questions in your own words (write as Python comments in your file): (a) What is a domain name and why do we need it instead of just using IP addresses? (b) Explain the DNS resolution process step by step — what happens from the moment you type google.com to the moment the page loads. (c) What is the IP address 8.8.8.8? What is 1.1.1.1? What type of servers are these? (d) If your internet stops working, how can you check and fix the DNS? Which command shows your network info? (e) What is the difference between HTTP and HTTPS?

```
# Save as: section_a.py

# Answer each part as a comment below:


# (a) Domain name explanation:

#


# (b) DNS resolution steps:

#  Step 1:

#  Step 2:

#  Step 3:

#  Step 4:


# (c) 8.8.8.8 is: ___________   1.1.1.1 is: ___________


# (d) Command to check DNS: ___________   Fix: ___________


# (e) HTTP vs HTTPS:

#
```

■ Hint: Review the Day 2 notes. DNS = Domain Name System. The resolver translates names to IPs.

Q2. HTTP Methods & Status Codes

THEORY 5 pts

Complete the table by filling in the missing descriptions. Then write a Python comment explaining a real-world example for each HTTP method (e.g. GET = loading your Twitter feed).

```
# Fill in the blanks:


# HTTP Method  |  Purpose                    |  Your real-world example
# -------------|-----------------------------|--------------------------
# GET          |  _____________________________  |
# POST         |  _____________________________  |
# PUT          |  _____________________________  |
# DELETE       |  _____________________________  |


# Status Code  |  Meaning
# -------------|---------------------------------
# 200          |  _____________________________
# 404          |  _____________________________
# 500          |  _____________________________
# 401          |  _____________________________


# What is a Request Body? Give an example in JSON format:
# What is a Response Body? Give an example:
```

■ Hint: GET = read/fetch. POST = create/send. PUT = update. DELETE = remove.

Q3. API Call — GitHub Profile

EASY 5 pts

Using the requests library, call the GitHub API for YOUR username. Extract and print the following 6 fields in a clean formatted output. Use .get() with default values so your code never crashes even if a field is missing.

```
import requests


url = 'https://api.github.com/users/YOUR_USERNAME'

data = requests.get(url).json()


# Print these 6 fields with labels:

# 1. login (username)

# 2. name

# 3. public_repos

# 4. followers

# 5. location  (print 'Not set' if None)

# 6. bio       (print 'No bio' if None)


# BONUS: If followers > 50, print 'Influencer!'

#        If public_repos > 10, print 'Active developer!'
```

■ Hint: Use data.get('location', 'Not set'). The second argument is the fallback default.

Section B — Python Basics

Days 3–4 | 15 pts

Q4. Data Types & type()

EASY

4 pts

For each line of code below, write what type() will return AND predict the output before running it. Then run it to check yourself. Write your predictions as comments.

```
# Save as: section_b.py

# Predict the TYPE and VALUE before running:


a = 42                      # type: _____  value: _____

b = 3.14                    # type: _____  value: _____

c = "hello"                 # type: _____  value: _____

d = True                    # type: _____  value: _____

e = int("99")               # type: _____  value: _____

f = str(100)                # type: _____  value: _____

g = float("3.5")            # type: _____  value: _____

h = int("hello")            # type: _____  what happens? ___


# Now run these and check:

print(type(a), a)

print(type(b), b)

print(type(c), c)

print(type(d), d)

print(type(e), e)

print(type(f), f)

print(type(g), g)
```

■ Hint: int('hello') will raise a ValueError — that is expected. Write what error you got.

Q5. Personal Details Program

EASY

5 pts

Write a program that asks the user for their name, age, city, and favourite programming language. Store each in a variable with the correct data type. Then print a properly formatted introduction sentence using string concatenation AND f-strings (do both).

```
# Take these inputs from the user:

name = input('Enter your name: ')

age  = int(input('Enter your age: '))

# ... add city and fav_language


# Method 1 — string concatenation using +

# print('Hi, my name is ' + ...)


# Method 2 — f-string

# print(f'Hi, my name is {name}...')


# Also print:

# - In 5 years you will be X years old

# - Your name has X characters (use len())
```

■ Hint: f-strings: f'Hello {name}' — the variable goes inside the curly braces. len(name) gives the length.

Q6. Type Conversion Errors

MEDIUM 6 pts

Each code block below has a bug related to type conversion or concatenation. Identify the error, explain WHY it happens, and fix it. There are 4 bugs total.

```
# BUG 1 — find and fix:

print("My age is: " + 25)

# Error: ___________  Fix: ___________


# BUG 2 — find and fix:

age = input("Age: ")

print("Next year: " + age + 1)

# Error: ___________  Fix: ___________


# BUG 3 — find and fix:

x = int("three")

# Error: ___________  Fix: ___________


# BUG 4 — find and fix:

num = input("Enter number: ")

result = num * 2

print("Double:", result)  # user types 5, but prints 55 not 10

# Error: ___________  Fix: ___________
```

■ Hint: Type errors usually happen when you mix str and int with +. Always convert before operating.

Section C — Operators & Conditionals

Days 4–5 | 15 pts

Q7. Operator Predict-the-Output

EASY

4 pts

WITHOUT running the code, predict each output. Write your answer in the blank. Then run it and mark ✓ if correct or ✗ and write the actual output.

```
# Save as: section_c.py


a, b = 17, 5

print(a + b)      # Predict: ___  Actual: ___

print(a - b)      # Predict: ___  Actual: ___

print(a * b)      # Predict: ___  Actual: ___

print(a / b)      # Predict: ___  Actual: ___

print(a // b)     # Predict: ___  Actual: ___

print(a % b)      # Predict: ___  Actual: ___

print(a ** b)     # Predict: ___  Actual: ___


print(a == b)     # Predict: ___  Actual: ___

print(a != b)     # Predict: ___  Actual: ___

print(a > b)      # Predict: ___  Actual: ___

print(a >= 17)    # Predict: ___  Actual: ___


print(a > 10 and b < 10)   # Predict: ___  Actual: ___

print(a > 20 or b < 10)    # Predict: ___  Actual: ___

print(not (a == b))        # Predict: ___  Actual: ___
```

■ Hint: 17 // 5 = 3 (floor). 17 % 5 = 2 (remainder). 17**5 = 1419857.

Q8. Smart Grade & Category System

MEDIUM 5 pts

Write a program that takes a student's marks as input and prints: their grade (A-F), their category (Distinction/Merit/Pass/Fail), and a personalised motivational message for each grade. Also validate that marks are between 0 and 100.

```
marks = int(input('Enter marks (0-100): '))


# Validation
if marks < 0 or marks > 100:
    print('Invalid marks')
else:
    # Grade: A(90-100), B(75-89), C(60-74), D(45-59), F(<45)
    # Category: Distinction(A), Merit(B), Pass(C/D), Fail(F)
    # Message: e.g. A → 'Outstanding! Keep it up!'
    #              F → 'Don't give up! Practice more.'
    pass  # replace with your code
```

■ Hint: Use elif for the grade chain. Add a separate if/elif for the category. Messages should be encouraging.

Q9. Logical Operators — Predict the Flow

MEDIUM 6 pts

This is a tricky code-tracing exercise. For each scenario below, trace through the code manually and write EXACTLY which print statement will execute and why. Then run it to verify.

```
username = 'admin'

password = '1234'

secret   = '7777'


# Scenario 1: user types admin / wrong / 7777

# Which line executes? ___  Why? ___


# Scenario 2: user types wrong / 1234 / 7777

# Which line executes? ___  Why? ___


# Scenario 3: user types admin / 1234 / 7777

# Which line executes? ___  Why? ___


# The code:

u = input('Username: ')

p = input('Password: ')

s = input('Secret: ')


if u == username and p == password and s == secret:

    print('Welcome!')

elif u != username:

    print('Wrong username')

elif p != password:

    print('Wrong password')

else:

    print('Wrong secret code')
```

■ Hint: Remember: with 'and', if the FIRST condition is False, Python does NOT check the rest — it already knows the result is False.

Section D — Loops

Day 5 | 15 pts

Q10. range() Mastery

EASY

4 pts

Write the range() call that produces each output. Then verify by running it.

```
# Save as: section_d.py


# Output: 1 2 3 4 5 6 7 8 9 10
# range(___________)


# Output: 0 2 4 6 8 10
# range(___________)


# Output: 10 9 8 7 6 5 4 3 2 1
# range(___________)


# Output: 5 10 15 20 25 30
# range(___________)


# Output: 100 90 80 70 60 50
# range(___________)


# How many times does this loop run?
for i in range(3, 50, 7):
    pass
# Answer: ___  Last value of i: ___
```

■ Hint: Remember: range(start, stop, step). The stop value is EXCLUDED. Negative step goes backwards.

Q11. Loop Patterns

MEDIUM

5 pts

Produce each of the following outputs using loops. Each pattern should be a separate loop.

```
# Pattern 1 — sum of 1 to 100 using a while loop

# Expected: Sum = 5050


# Pattern 2 — print all numbers 1-50 divisible by both 3 AND 5

# Expected: 15  30  45


# Pattern 3 — countdown from 20, skip multiples of 4, stop at 0

# Expected: 20 19 18 17 15 14 13 11 10 9 7 6 5 3 2 1

# (16, 12, 8, 4 are skipped. Stop AT 0, don't print it.)


# Pattern 4 — number pyramid

# Expected:

#   1

#   1 2

#   1 2 3

#   1 2 3 4

#   1 2 3 4 5
```

■ Hint: Pattern 1: total = 0, while i <= 100. Pattern 3: use both continue and break. Pattern 4: nested loops — outer for rows, inner for columns.

Q12. break, continue & while..else

HARD

6 pts

Answer three parts about loop control.

```
# PART A — Trace this code. Write the EXACT output:

for i in range(1, 11):

    if i % 3 == 0:

        continue

    if i == 8:

        break

    print(i, end=' ')

# Output: ___


# PART B — Fix the infinite loop (don't change the while condition):

i = 1

while i <= 5:

    print(i)

    # something is missing here


# PART C — Write a program using while..else:

# Ask the user to guess a secret word (hardcode it as 'python').

# Give them 3 attempts. If correct → break + print 'Correct!'

# If all 3 fail → else block prints 'The word was: python'
```

■ Hint: PART A: continue skips, break exits. Trace i=1,2,3(skip),4,5,6(skip),7,8(break). PART C: while..else — else runs only if no break.

Section E — Data Structures

Days 5–6 | 20 pts

Q13. List — Shopping Cart

EASY

5 pts

Build a simple shopping cart using a list. Perform all operations in order and print the cart after every step.

```
# Save as: section_e.py


cart = ["milk", "bread", "eggs"]


# 1. Add 'butter' to the end

# 2. Add 'juice' at position 1 (index 1)

# 3. Remove "bread" by value

# 4. Print how many items are in the cart

# 5. Sort the cart alphabetically and print

# 6. Print the LAST item without using index -1 (use len())

# 7. Check if "eggs" is in cart → print True/False

# 8. Pop the last item and print what was removed

# 9. Print the final cart
```

■ Hint: For #6: use cart[len(cart)-1]. The 'in' keyword checks membership: 'eggs' in cart.

Q14. Tuple vs List — Know the Difference

THEORY

4 pts

This question tests your understanding of when to use each data structure. Answer as comments in your code file.

```
# Q1. Why can't you do t.append(5) if t is a tuple?

# Answer:


# Q2. What is the output of this code and WHY?

t = (10, 20, 30, 20, 10)

print(t.count(20))    # Output: ___  Why: ___

print(t.index(30))    # Output: ___  Why: ___

print(max(t))         # Output: ___

print(min(t))         # Output: ___


# Q3. In Django, tuples are used for things like URL patterns.

# Give 2 other real-world examples where you'd use a tuple

# instead of a list:

# Example 1:

# Example 2:


# Q4. Convert this list to a tuple, then try to change the first element.

# What error do you get?

nums = [1, 2, 3, 4, 5]
```

■ Hint: Tuple = immutable (frozen list). Use when data should NOT change. count() counts occurrences, index() gives position.

Q15. Set — Deduplication & Operations

MEDIUM

5 pts

Work with sets for two real-world tasks.

```
# TASK 1 — Remove duplicates from exam results

scores = [88, 72, 95, 88, 61, 72, 100, 88, 95, 61]

# Convert to set to get unique scores

# Print: total scores submitted, unique scores, highest, lowest


# TASK 2 — Course enrollment analysis

python_students = {"alice","bob","carol","dave","eve"}

django_students  = {"carol","dave","frank","grace","eve"}


# Find and print:

# 1. Students enrolled in BOTH courses

# 2. All unique students across both courses

# 3. Students ONLY in Python (not Django)

# 4. Students ONLY in Django (not Python)

# 5. Is "alice" in Django? Print True/False

# 6. Add "harry" to Python course

# 7. Remove 'bob' from Python course safely (use discard)
```

■ Hint: Set difference: A - B gives items in A but not B. B - A gives items in B but not A. Always use discard() instead of remove() to avoid KeyError.

Q16. Dictionary — Student Management System

HARD

6 pts

Build a mini student management system using a dictionary. Complete all tasks in order.

```
students = {"rahul":90, "john":55, "anita":72, "priya":38, "sara":85}


# 1. Print all student names (keys only)

# 2. Print all marks (values only)

# 3. Print all name-mark pairs using .items()

# 4. Add a new student: 'ram' with marks 67

# 5. Update john's marks to 78 (he improved!)

# 6. Remove 'priya' from the dictionary

# 7. Safe lookup: get marks for 'meera' (not in dict)

#    Use .get() with 'Not enrolled' as fallback

# 8. Find and print the student with the HIGHEST marks

#    (use a for loop to compare — do not use max())

# 9. Count how many students scored above 70

# 10. Print a grade report: name → mark → grade (A/B/C/D/F)
```

■ Hint: For #8: top_student = '' and top_marks = 0, then loop and compare. For #9: use a counter variable and increment inside an if.

Section F — Functions

Day 6 | 10 pts

Q17. Function Anatomy — Fill in the Blanks

EASY

3 pts

Complete each function correctly and call it with appropriate arguments.

```
# Save as: section_f.py


# 1. Complete this function — returns the square of a number

def square(n):

    ____________


print(square(5))    # Expected: 25

print(square(12))   # Expected: 144


# 2. Complete this function — returns the larger of two numbers

def find_max(a, b):

    ____________


print(find_max(10, 20))  # Expected: 20

print(find_max(99, 3))   # Expected: 99


# 3. Complete this — returns True if a number is even, False if odd

def is_even(n):

    ____________


print(is_even(4))   # Expected: True

print(is_even(7))   # Expected: False
```

■ Hint: Use return inside the function. For is_even: return n % 2 == 0 — this expression itself is a boolean.

Q18. Reusability — Function Library

MEDIUM

7 pts

Build a small function library. Each function should be reusable. Then use them together in a final program.

```
# Write all 5 functions:


# 1. celsius_to_fahrenheit(c) → returns (c * 9/5) + 32

# 2. count_vowels(text) → returns count of a,e,i,o,u in text

# 3. reverse_string(text) → returns text reversed

# 4. is_palindrome(text) → returns True if text == text reversed

# 5. word_count(sentence) → returns number of words


# Then call them to produce this output:

# celsius_to_fahrenheit(0)   → 32.0

# celsius_to_fahrenheit(100) → 212.0

# count_vowels("Hello World") → 3

# reverse_string("python")   → nohtyp

# is_palindrome("racecar")   → True

# is_palindrome("hello")     → False

# word_count("I love Python programming") → 4
```

■ Hint: reverse_string: return text[::-1] (slicing with step -1). is_palindrome: compare text.lower() == text.lower()[::-1]. count_vowels: loop through text, check 'if char in "aeiouAEIOU"'.

Section G — Mini Project

Days 2–6 | 10 pts

Q19. Student Report Card Generator

PROJECT 10 pts

Build a complete program combining ALL concepts from Days 3-6. Save as report_card.py.

```
# 1. DICTIONARY — 5+ students, each with maths/science/english marks
# students = {"Rahul":{"maths":88,"science":75,"english":92}, ...}


# 2. def calculate_average(marks_dict) → returns average of 3 subjects
# 3. def get_grade(average) → returns A/B/C/D/F
# 4. def get_remark(grade) → returns motivational message


# 5. for name, subjects in students.items():
#        avg = calculate_average(subjects)
#        grade = get_grade(avg)
#        remark = get_remark(grade)
#        print formatted report here


# 6. Collect all averages in a list → find class avg, highest, lowest
# 7. Collect all grades in a SET → print unique grades awarded
```

■ Hint: Start with ONE student working end-to-end, then add the loop. This combines dictionary + list + set + functions + loops all at once.